



UMCS

UNIwersytet Marii Curie-Skłodowskiej
w Lublinie

Wydział Matematyki, Fizyki i Informatyki

Kierunek: **informatyka**

Daniel Wiszniowski

nr albumu: 318635

Projekt i realizacja bezakumulatorowych czujników klimatycznych zasilanych słonecznie

Design and development of battery-free
solar-powered climate sensors

Praca licencjacka

napisana w Katedrze Oprogramowania Systemów Informatycznych

Instytutu Informatyki i Matematyki UMCS

pod kierunkiem **dr hab. Beaty Byliny, prof. UMCS**

Lublin 2026

Spis treści

Wstęp	5
1 Wprowadzenie do tematyki	7
1.1 Czujniki klimatyczne	7
1.2 Moduły pomiarowe	7
1.3 Zasilanie czujników	9
2 Programowanie systemów wbudowanych	11
2.1 Charakterystyka układów sterujących	11
2.2 Języki programowania	12
2.2.1 Język C	12
2.2.2 Język C++	13
2.2.3 MicroPython	13
2.2.4 Język Rust	13
2.2.5 Podsumowanie i porównanie cech języków	14
2.3 Obsługa magistrali	14
3 Założenia projektowe	15
3.1 Komunikacja radiowa	15
3.2 Zastosowanie paneli fotowoltaicznych	15
3.3 Zastosowanie superkondensatorów	15
3.4 Niski pobór mocy	15
4 Część sprzętowa	17
4.1 Podstawowe elementy układu	17
4.1.1 Sekcja ładowania superkondensatorów	17
4.1.2 Układ regulacji napięcia	18
4.1.3 Mikrokontroler ATmega328P	19
4.1.4 Magistrala I^2C	21
4.1.5 Układ komunikacji radiowej	22

4.2	Elementy pomiarowe	22
4.2.1	Barometr, higrometr i termometr	22
4.2.2	Moduł z licznikiem Geigera-Müllera	22
4.2.3	Moduł pomiaru natężenia światła	23
4.3	Elementy dodatkowe	23
4.3.1	Magistrala 1-Wire	23
4.3.2	Wolne wyprowadzenia GPIO	23
5	Oprogramowanie	25
5.1	Język Rust w programowaniu mikrokontrolerów	25
5.2	Narzędzia i biblioteki	25
5.3	Obsługa elementów pomiarowych	25
5.3.1	Moduł AHT20 — czujnik temperatury i wilgotności względnej	25
5.3.2	Moduł BMP280	29
5.3.3	Moduł VEML7700	33
5.3.4	Moduł z licznikiem Geigera-Müllera	33
5.4	Komunikacja radiowa	33
5.4.1	Serializacja danych	33
5.4.2	Zabezpieczenie danych	33
5.4.3	Transmisja danych	33
6	Testy	35
6.1	Zasięg komunikacji radiowej	35
6.2	Pobór prądu	35
6.3	Czas pracy	35
6.4	Porównanie z rozwiązaniami komercyjnymi	35
	Podsumowanie	37
	Spis listingów	39
	Spis tabel	41
	Spis rysunków	43
	Bibliografia	45

Wstęp

Tu treść wstępu (który piszemy na końcu, po napisaniu całej pracy).

Rozdział 1

Wprowadzenie do tematyki

1.1 Czujniki klimatyczne

Współczesny rozwój technologiczny w połączeniu z postępującymi zmianami klimatu oraz rosnącą automatyzacją procesów przemysłowych i urządzeń domowych sprawiły, że precyzyjne obserwowanie i dokumentowanie parametrów środowiskowych stało się nieodłącznym elementem współczesnej inżynierii.

Głównym zadaniem czujników klimatycznych jest ciągła i bezobsługowa akwizycja danych dotyczących parametrów atmosfery w wybranym obszarze. Dane te są kluczowe w wielu dziedzinach życia gospodarczego i społecznego. W rolnictwie precyzyjnym (ang. *smart agriculture*) pozwalają na optymalizację nawadniania i ochrony upraw [TODO: citation needed], w aglomeracjach miejskich służą do monitorowania smogu i poziomu zanieczyszczeń powietrza, natomiast w systemach zarządzania budynkami (ang. *Building Management Systems*) stanowią podstawę sterowania procesami klimatyzacji, wentylacji i ogrzewania.

[TODO: Może jeszcze jeden akapit? Praca Wołodko? Modularność?]

1.2 Moduły pomiarowe

Do podstawowych wielkości fizycznych rejestrowanych przez czujniki klimatyczne zalicza się przede wszystkim temperaturę, wilgotność względną oraz ciśnienie atmosferyczne. Do pomiaru tych wielkości coraz rzadziej wykorzystuje się bezpośrednio połączone z układem czujniki analogowe — są one wypierane przez moduły integrujące elementy pomiarowe, dedykowane przetworniki analogowo-cyfrowe (ADC, ang. *analog-to-digital converter*) oraz fabryczne obwody kalibracyjne w jednej obudowie. Rozwiązanie takie pozwala wyeliminować problem zakłóceń elektromagnetycznych indukowanych w przewodach sygnałowych. Ponadto projektanci modułów implementują

algorytmy linearyzacji charakterystyk sensorów oraz cyfrowe filtry szumów, co umożliwia wygodny odczyt danych bezpośrednio przez mikrokontroler.

Na rynku dostępnych jest szereg modułów realizujących pomiary klimatyczne, różniących się zarówno mierzonymi wielkościami fizycznymi, jak i osiąganą dokładnością, zakresem pomiarowym oraz stopniem integracji. Wśród popularnych modułów pomiarowych temperatury i wilgotności wyróżnić można przede wszystkim rodzinę DHT (DHT11, DHT22) firmy AOSONG, charakteryzującą się niskim kosztem i szeroką dostępnością, lecz ograniczoną dokładnością oraz komunikacją przez dedykowany, jednobitowy protokół szeregowy. Wyższą klasę dokładnościową reprezentują moduły szwajcarskiej firmy Sensirion — seria SHT3x (SHT30, SHT31, SHT35) oraz nowsza seria SHT4x (SHT40, SHT41, SHT45), komunikujące się przez interfejs I²Ci oferujące dokładność pomiaru temperatury na poziomie $\pm 0,2^\circ\text{C}$. W obszarze pomiaru ciśnienia atmosferycznego dominuje oferta firmy Bosch Sensortec — od popularnego BMP180, przez powszechnie stosowany BMP280, po nowsze BMP388 i BMP390 charakteryzujące się wyższą rozdzielczością. Firma Bosch oferuje również moduły z rodziny BME (BME280, BME680, BME688), łączące pomiar ciśnienia, temperatury i wilgotności w jednej obudowie, przy czym moduły BME680 i BME688 wyposażone są dodatkowo w czujnik gazów lotnych (VOC). Do pomiaru natężenia oświetlenia w zakresie widzialnym najczęściej stosuje się moduły BH1750 firmy ROHM oraz VEML7700 firmy Vishay, oba komunikujące się przez interfejs I²C.

Zestawienie wybranych, reprezentatywnych modułów dostępnych na rynku wraz z ich podstawowymi parametrami przedstawiono w tabeli 1.1.

Tabela 1.1: Zestawienie wybranych modułów pomiarowych dostępnych na rynku

Moduł	Wielkości mierzone	Interfejs	Uwagi
DHT22 (AM2302)	T, RH	1-Wire	niski koszt
AHT20*	T, RH	I ² C	—
SHT31	T, RH	I ² C	wysoka stabilność
SHT40	T, RH	I ² C	najnowsza generacja
DS18B20	T	1-Wire	wodoodporny wariant
BMP180	T, p	I ² C	przestarzały
BMP280*	T, p	I ² C/SPI	niski pobór mocy
BME280	T, RH, p	I ² C/SPI	3 wielkości
BME688	T, RH, p, VOC	I ² C/SPI	detekcja gazów
BH1750	E _v	I ² C	do 65 535 lx
VEML7700*	E _v	I ² C	do 120 000 lx

T – temperatura, RH – wilgotność względna, p – ciśnienie atmosferyczne, E_v – natężenie oświetlenia; * moduły zastosowane w niniejszej pracy.

Do realizacji systemu opisanego w niniejszej pracy wybrano moduły AHT20, BMP280 oraz VEML7700. Kryterium doboru była dostępność szczegółowej dokumentacji technicznej w postaci kart katalogowych producenta, umożliwiającej samodzielną implementację sterowników programowych. Wszystkie trzy moduły komunikują się przez interfejs I²C, co upraszcza schemat połączeń elektrycznych — wymagają one jedynie wspólnej magistrali dwuprzewodowej. Funkcjonalnie moduły te wzajemnie się uzupełniają: AHT20 realizuje pomiar temperatury i wilgotności względnej, BMP280 dostarcza danych o ciśnieniu atmosferycznym, natomiast VEML7700 rozszerza zestaw pomiarowy o natężenie oświetlenia widzialnego. Implementacja warstwy komunikacyjnej dla każdego z modułów została wykonana samodzielnie na podstawie specyfikacji protokołu I²C oraz rejestrów opisanych w dokumentacji producenta, co szczegółowo omówiono w rozdziale 5.3.

1.3 Zasilanie czujników

Sposób zasilania czujników klimatycznych jest uzależniony od miejsca ich docelowego zastosowania. W przypadku montażu wewnętrznego możliwe jest wykorzystanie sieciowego zasilacza impulsowego o napięciu wyjściowym dobranym do wymagań układu, jak również jednorazowych baterii alkalicznych. W przypadku montażu zewnętrznego doprowadzenie zasilania przewodem bywa często niepraktyczne lub niemożliwe. Alternatywę stanowią baterie jednorazowe, jednak wymagają one okresowej wymiany. Rozwiązaniem przyjętym w niniejszym projekcie jest zasilanie z paneli fotowoltaicznych.

Zastosowanie ogniw fotowoltaicznych wymaga jednak uzupełnienia układu o magazyn energii umożliwiający podtrzymanie pracy czujnika w okresach braku nasłonecznienia. Powszechnie stosowanym rozwiązaniem są ogniwa litowo-jonowe, których wybór uzasadnia malejący koszt, szeroka dostępność rynkowa oraz rosnąca gęstość energetyczna (ang. *energy density*) [10][TODO: Nowsze źródło]. Ich wadami są jednak stosunkowo złożony układ ładowania i zabezpieczenia przed nadmiernym rozładowaniem oraz ograniczona odporność na duże wahania temperatur [11].

Wymienionych ograniczeń pozbawione są superkondensatory. Podobnie jak ogniwa litowo-jonowe, charakteryzują się one malejącym kosztem i rosnącą dostępnością, a ponadto wykazują znacznie lepszą odporność na wahania temperatury [11] oraz nie wymagają ograniczenia głębokości rozładowania. Układ ładowania sprowadza się w ich przypadku wyłącznie do zabezpieczenia przed przekroczeniem maksymalnego napięcia na okładkach, co omówiono w podrozdziale 4.1.1. Cechy te czynią superkondensatory rozwiązaniem szczególnie odpowiednim dla zewnętrznych czujników zasilanych fotowoltaicznie. Istotną wadą superkondensatorów pozostaje jednak znacznie mniej-

szą gęstość energetyczną w porównaniu z ogniwami litowo-jonowymi, co ogranicza ich zastosowanie do urządzeń o niskim poborze mocy.

Rozdział 2

Programowanie systemów wbudowanych

System wbudowany (ang. *embedded system*) stanowi wyspecjalizowany system komputerowy, będący integralną częścią większego układu mechanicznego lub elektronicznego. W przeciwieństwie do komputerów osobistych ogólnego przeznaczenia (ang. *general-purpose computers*), systemy te są projektowane do realizacji ściśle zdefiniowanych zadań, co pozwala na optymalizację pod kątem wydajności, kosztów oraz zużycia energii. Współcześnie systemy wbudowane znajdują zastosowanie w szerokim spektrum urządzeń — od sprzętu gospodarstwa domowego (AGD), po zaawansowane systemy sterowania w przemyśle i motoryzacji [7].

2.1 Charakterystyka układów sterujących

Kluczowym elementem decyzyjnym systemu wbudowanego jest układ scalony o wysokim stopniu integracji — mikrokontroler lub mikroprocesor.

Mikroprocesor (MPU) integruje jednostkę centralną (CPU) oraz rejestry danych, jednak do poprawnej pracy wymaga zewnętrznych komponentów, takich jak pamięć operacyjna (RAM), pamięć nieulotna oraz kontrolery peryferiów. Z kolei mikrokontroler (MCU) jest układem autonomicznym, który w jednej obudowie zawiera procesor, pamięć oraz układy wejścia-wyjścia. Ze względu na wysoką integrację, niski koszt implementacji oraz uproszczoną architekturę sprzętową, w niniejszym projekcie zdecydowano się na wykorzystanie mikrokontrolera.

Wybór konkretnego układu zależy od specyfiki projektu. Do najpopularniejszych rodzin mikrokontrolerów należą:

- **Rodzina AVR (Atmel/Microchip):** 8-bitowe układy charakteryzujące się prostotą programowania, spopularyzowane dzięki platformie Arduino. Są opty-

malne dla prostych systemów sterowania.

- **Rodzina ESP (Espressif Systems):** Układy ESP8266 oraz ESP32, które zre-wolucjonizowały rynek dzięki zintegrowanym modułom łączności bezprzewodowej Wi-Fi oraz Bluetooth, przy zachowaniu bardzo korzystnego stosunku ceny do wydajności.
- **STM32 (STMicroelectronics):** 32-bitowe mikrokontrolery oparte na architekturze ARM Cortex-M, oferujące wysoką moc obliczeniową i bogaty zestaw peryferiów, powszechnie stosowane w rozwiązaniach profesjonalnych.

Ze względu na niski stopień skomplikowania projektu wybrany został procesor **ATmega328p** firmy Atmel, którego dokładniejszy opis przedstawiono w podrozdziale [4.1.3](#).

2.2 Języki programowania

Wybór odpowiedniego języka programowania ma kluczowe znaczenie dla efektywności procesu wytwarzania oprogramowania, stabilności systemu oraz optymalnego wykorzystania ograniczonych zasobów sprzętowych mikrokontrolera. W architekturze systemów wbudowanych tradycyjnie dominują języki niskopoziomowe, jednak rozwój technologii oraz wzrost mocy obliczeniowej układów scalonych przyczyniły się do popularyzacji alternatywnych rozwiązań.

2.2.1 Język C

Język C od dekad pozostaje standardem branżowym w dziedzinie systemów wbudowanych. Jego główną zaletą jest wysoka wydajność, minimalny narzut pamięciowy oraz bezpośredni dostęp do rejestrów procesora i zasobów sprzętowych. Kod źródłowy kompilowany jest bezpośrednio do kodu maszynowego, co zapewnia przewidywalność czasu wykonania (ang. *deterministic execution time*). Ponadto, producenci mikrokontrolerów dostarczają dedykowane biblioteki HAL (ang. *Hardware Abstraction Layer*) najczęściej właśnie w języku C.

Główną wadą języka C jest brak wbudowanych mechanizmów bezpieczeństwa pamięci. Problemy takie jak przepełnienie bufora (ang. *buffer overflow*), wycieki pamięci (ang. *memory leak*) czy usterki typu *use-after-free* muszą być kontrolowane bezpośrednio przez programistę, co zwiększa ryzyko wystąpienia krytycznych błędów w działaniu urządzenia.

2.2.2 Język C++

Język C++ wprowadza do świata systemów wbudowanych paradygmat programowania obiektowego (OOP) oraz mechanizmy abstrakcji, takie jak szablony (ang. *templates*) czy polimorfizm. Pozwala to na lepszą strukturyzację i modularność kodu, co jest szczególnie istotne w złożonych projektach.

W kontekście mikrokontrolerów stosuje się zazwyczaj ograniczony podzbiór języka C++ (tzw. *Embedded C++*). Często rezygnuje się z dynamicznej alokacji pamięci (operator *new/delete*) oraz obsługi wyjątków (ang. *exceptions*), ponieważ mechanizmy te wprowadzają nieprzewidywalność czasu wykonania programu i mogą prowadzić do fragmentacji pamięci RAM.

2.2.3 MicroPython

MicroPython to zoptymalizowana implementacja języka Python 3, stworzona z myślą o pracy na nowoczesnych mikrokontrolerach (np. ESP32, STM32). Jego kluczową zaletą jest niezwykle wysoki poziom abstrakcji, prostota składni oraz interpretacja kodu w czasie rzeczywistym, co drastycznie skraca czas tworzenia prototypów.

Niestety, jako język interpretowany, MicroPython charakteryzuje się znacznie niższą wydajnością i większym zużyciem pamięci RAM oraz Flash w porównaniu do języków kompilowanych. Obecność mechanizmu odśmiecania pamięci (ang. *Garbage Collector*) wyklucza go z zastosowań w systemach czasu rzeczywistego (RTOS), gdzie wymagany jest rygorystyczny determinizm czasowy.

2.2.4 Język Rust

Język Rust reprezentuje nowoczesne podejście do programowania systemowego, łącząc wydajność i niskopoziomą kontrolę znaną z języków C/C++ z nowoczesnymi mechanizmami bezpieczeństwa. W kontekście systemów wbudowanych, kluczową cechą Rusta jest koncepcja zarządzania pamięcią oparta na systemie własności (ang. *ownership*) oraz cyklach życia obiektów (ang. *lifetimes*). Walidacja tych reguł odbywa się na etapie kompilacji (ang. *compile-time*), co eliminuje całą klasę błędów związanych z dostępem do pamięci bez wprowadzania narzutu w czasie wykonywania programu (ang. *zero-cost abstractions*) oraz bez użycia mechanizmu *Garbage Collector*.

Dodatkowo, ekosystem języka Rust oferuje zaawansowane narzędzia menedżera pakietów *cargo*, co ułatwia zarządzanie zależnościami oraz automatyzację testów. Choć próg wejścia w ten język jest wysoki ze względu na restrykcyjny kompilator, bezpieczeństwo kodu i odporność na błędy współbieżności (ang. *data races*) sprawiają, że

staje się on coraz częstszym wyborem w projektowaniu niezawodnych systemów wbudowanych.

2.2.5 Podsumowanie i porównanie cech języków

W celu uzasadnienia wyboru narzędzi programistycznych w niniejszej pracy, w tabeli 2.1 zestawiono kluczowe cechy omawianych języków programowania.

Tabela 2.1: Porównanie języków programowania w systemach wbudowanych.

Cecha / Kryterium	Język C	Język C++	MicroPython	Język Rust
Wydażność obliczeniowa	Bardzo wysoka	Bardzo wysoka	Niska	Bardzo wysoka
Zarządzanie pamięcią	Ręczne	Ręczne / Obiektowe	Garbage Collector	Statyczne
Bezpieczeństwo pamięci	Brak	Brak / Ograniczone	Wysokie	Całkowicie
Poziom abstrakcji	Niski	Średni/Wysoki	Bardzo wysoki	Wysoki
Determinizm czasowy	Tak	Tak (z ograniczeniami)	Nie	Tak

Jak wynika z powyższego porównania, język Rust stanowi optymalny kompromis pomiędzy bezkompromisową wydajnością i determinizmem cechującym język C, a nowoczesnymi mechanizmami bezpieczeństwa i wysokim poziomem abstrakcji. Z tego względu został on wybrany jako główne narzędzie programistyczne w realizacji projektu praktycznego.

2.3 Obsługa magistrali

[Opisz czym jest I2C z perspektywy kodu]

Rozdział 3

Założenia projektowe

3.1 Komunikacja radiowa

3.2 Zastosowanie paneli fotowoltaicznych

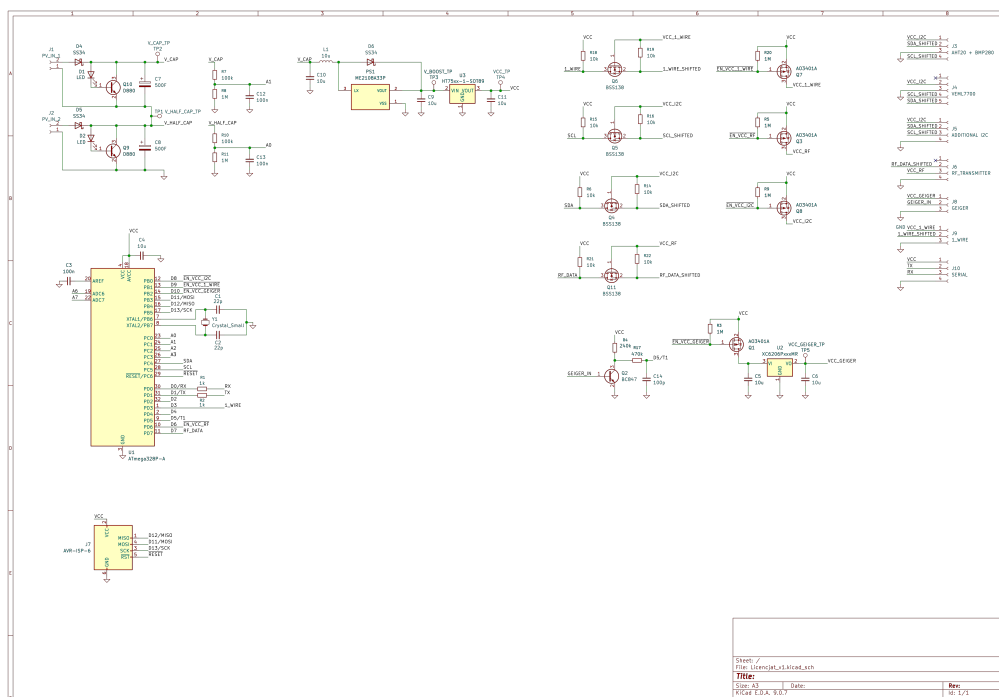
3.3 Zastosowanie superkondensatorów

3.4 Niski pobór mocy

Część sprzętowa

4.1 Podstawowe elementy układu

Schemat elektryczny, widoczny na rysunku 4.1, został podzielony na segmenty pełniące konkretne funkcje. [TODO: lepszy rysunek]



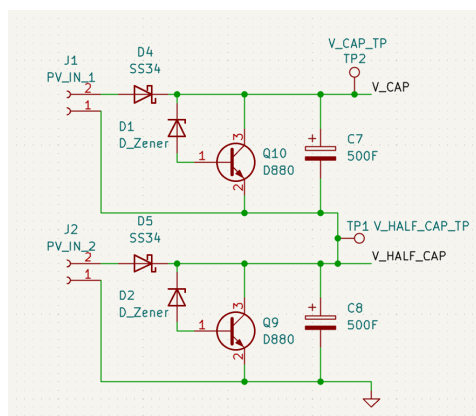
Rysunek 4.1: Schemat elektryczny części sprzętowej projektu.

4.1.1 Sekcja ładowania superkondensatorów

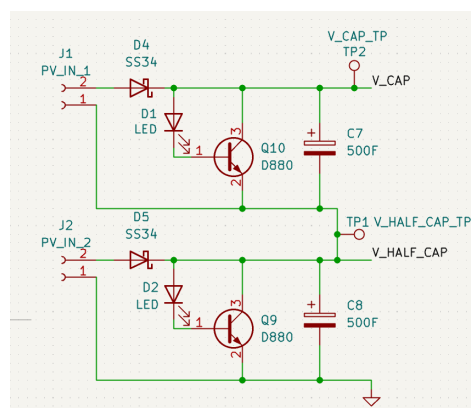
Zastosowanie paneli fotowoltaicznych jako jedyne go źródła energii do ładowania superkondensatorów wiąże się z określonymi ograniczeniami. Ze względu na zmienne

warunki nasłonecznienia, wynikające z czynników zewnętrznych, takich jak okresowe zacinienie panelu przez elementy otoczenia lub zmiany pory dnia, napięcie generowane na wyjściu panelu fotowoltaicznego nie ma charakteru stałego. W celu zapobieżenia wystąpieniu niepożądanych i nieodwracalnych zmian w strukturze superkondensatora konieczne jest ograniczenie napięcia ładowania do wartości nieprzekraczającej napięcia znamionowego [13], które w analizowanym układzie wynosi 2,7 V. W związku z powyższym napięcie pochodzące z ogniwa fotowoltaicznego jest regulowane za pomocą regulatora bocznikowego (ang. *shunt regulator*), zrealizowanego w postaci tranzystora zwierającego ze sobą wyjścia ogniwa oraz diody sterującej jego pracą. Dodatkowo, w celu zabezpieczenia panelu fotowoltaicznego przed przepływem prądu wstecznego, na jego dodatkim wyjściu zastosowano diodę Schottky’ego.

W początkowej wersji regulatora bocznikowego jako element sterujący pracą tranzystora zastosowano diodę Zenera. Jego schemat przedstawiono na rysunku 4.2. W wyniku przeprowadzonych testów rozwiązanie to zostało jednak zastąpione diodą LED (rysunek 4.3), co pozwoliło na zwiększenie stabilności pracy regulatora w zakresie prądu przewodzonego przez tranzystor. Porównanie parametrów obu rozwiązań przedstawiono w tabeli 4.1.



Rysunek 4.2: Sekcja ładowania superkondensatorów z użyciem diod zenera (D1, D2)



Rysunek 4.3: Sekcja ładowania superkondensatorów z użyciem diod LED (D1, D2)

4.1.2 Układ regulacji napięcia

Układ regulacji napięcia, przedstawiony na rysunku 4.4, składa się z 2 połączonych ze sobą części.

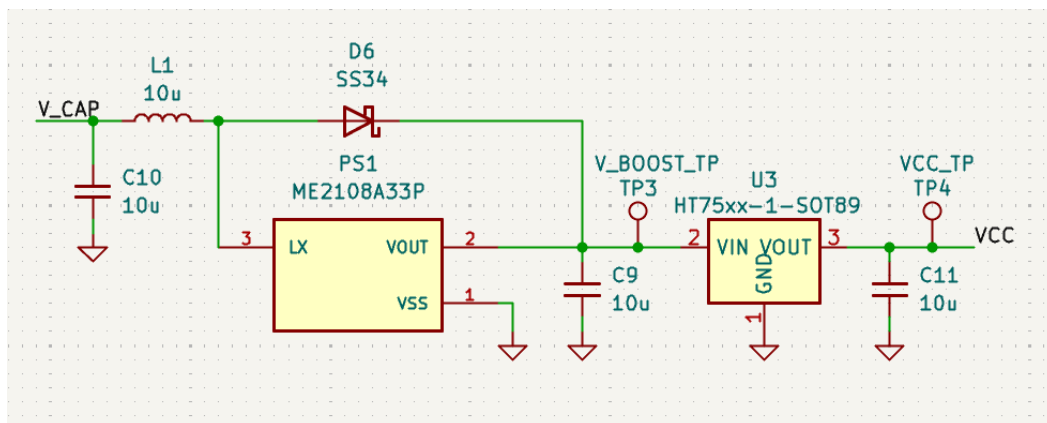
Pierwszą z nich jest przetwornica typu step-up, zbudowana w oparciu o układ ME210833P. Jej zastosowanie pozwala na zapewnienie zasilania gdy napięcie na połączonych szeregowo superkondensatorach spadnie poniżej 3,3V. Pozwoli to na rozłado-

Tabela 4.1: Napięcie wyjściowe U_{Wy} regulatora bocznikowego przy użyciu diody Zenera i diody LED, dla prądu emitera I_E tranzystora zwierającego. Do testu użyto zasilacza stałoprądowego.

I_E [mA]	U_{Wy} Zener [V]	U_{Wy} LED [V]
5	1,80	2,38
20	2,09	2,45
50	2,25	2,50
100	2,40	2,53
200	2,65	2,57
500	3,00	2,68

wanie ich do łącznego napięcia około 1V, co wynika z ograniczeń układu sterującego przetwornicy [12]. W momencie gdy napięcie wejściowe przetwornicy przekroczy 3,3V, zostanie ona pominięta, przez diodę Schottky’ego podłączoną pomiędzy jej wejściem i wyjściem.

Napięcie jest następnie ograniczane do wartości 3.3V z użyciem regulatora liniowego HT7333 o niskim spadku napięcia (rzędu 80mV [9]), i niskim prądzie spoczynkowym (o maksymalnej wartości 8 μ A [9]).



Rysunek 4.4: Układ regulacji napięcia oparty o przetwornicę step-up i regulator liniowy.

4.1.3 Mikrokontroler ATmega328P

Głównym elementem układu jest mikrokontroler ATmega328P, oparty o 8-bitowy procesor z architekturą AVR firmy Atmel, posiadający 32KB wbudowanej pamięci typu flash [6]. Najważniejszymi jego cechami, w kontekście tego projektu, są niski pobór mocy, możliwość uśpienia oraz wbudowane konwertery analogowo-cyfrowe [6]. Ze względu na popularność mikrokontrolerów rodziny AVR i wykorzystanie ich w platformach takich jak Arduino UNO [3] czy Arduino NANO [4], posiadają one rozbudo-

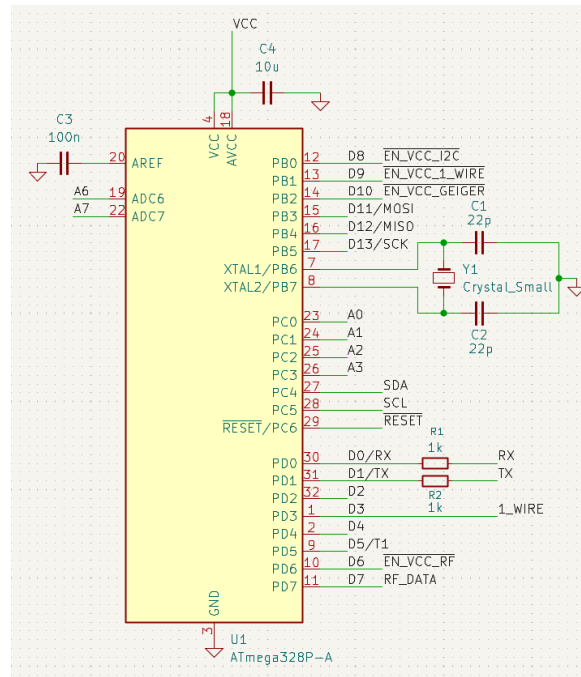
wany ekosystem zapewniający narzędzia potrzebne do ich oprogramowania.

Mikrokontroler został wyposażony w 3,6864 MHz oscylator kwarcowy. Wartość ta została wybrana ze względu na wygodę generowania standardowych prędkości UART dla których dzielniki, przy takiej częstotliwości, są liczbami całkowitymi [1].

Schemat połączeń mikrokontrolera widoczny jest na rysunku 4.5, a opis funkcji portów jest przedstawiony w tabeli 4.2.

Tabela 4.2: Opis funkcji wyprowadzeń mikrokontrolera. Typy portów: I/O - cyfrowy port wejścia/wyjścia, Th - podłączenie do wewnętrznego licznika n mikrokontrolera, ADC - wejście konwertera analogowo-cyfrowego.

Port	Przeznaczenie pinu	Funkcja
PB0	I/O	Sterowanie zasilaniem urządzeń magistrali I^2C .
PC4	I/O	Linia danych magistrali I^2C .
PC5	I/O	Linia sygnału zegarowego magistrali I^2C .
PB1	I/O	Sterowanie zasilaniem urządzeń magistrali 1Wire.
PD3	I/O	Linia danych magistrali 1Wire.
PD6	I/O	Sterowanie zasilaniem dla nadajnika radiowego
PD7	I/O	Linia danych nadajnika radiowego
PB2	I/O	Sterowanie zasilaniem dla modułu radiometrycznego.
PD3	T1	Zliczanie impulsów modułu radiometrycznego.
PC0	ADC	Badanie napięcia na 1 z 2 superkondensatorów.
PC1	ADC	Badanie napięcia połączonych szeregowo superkondensatorów.

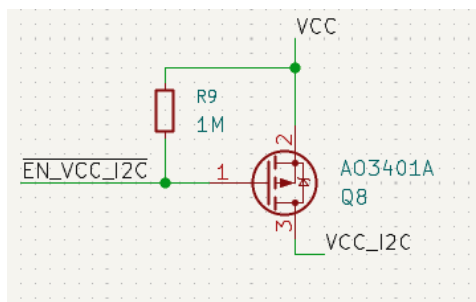


Rysunek 4.5: Mikrokontroler ATmega328P widoczny na schemacie projektu.

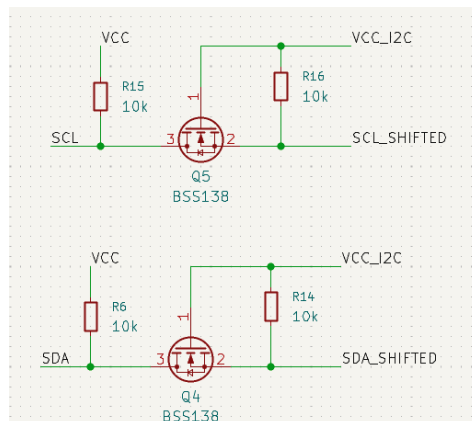
4.1.4 Magistrala I^2C

I^2C jest magistralą i protokołem synchronicznym umożliwiającym komunikację pomiędzy urządzeniami z użyciem 2 połączeń: zegarowego (oznaczanego jako SCL) oraz danych (oznaczanego jako SDA). Wykorzystane w tym celu wyprowadzenia mikrokontrolera widoczne są w tabeli 4.2. W projekcie użyte zostały 3 urządzenia używające tej magistrali: AHT20, BMP280 i VEML7700, które zostaną omówione w rozdziale 4.2.

Dodatkowo, w celu konserwacji energii, linia zasilania urządzeń tej magistrali może zostać odłączona poprzez podanie stanu wysokiego na tranzystor widoczny na rysunku 4.6. Aby zabezpieczyć niezasilone urządzenia magistral przed napięciem, mogącym się pojawić na liniach SDA i SCL, zostały zastosowane konwertery poziomów logicznych (ang. *level shifters*), pokazane na rysunku 4.7. Jednocześnie rezystory w nich zastosowane pełnią funkcję rezystorów podciągających dla linii SDA i SCL, wymaganych przez specyfikację standardu I^2C [14].



Rysunek 4.6: Tranzystor załączający linię zasilania urządzeń magistrali I^2C .



Rysunek 4.7: Konwertery poziomów logicznych dla linii SDA i SCL magistrali I^2C .

4.1.5 Układ komunikacji radiowej

Komunikacja radiowa odbywa się poprzez bezpośrednie sterowanie linią danych modułu nadajnika radiowego. Odpowiedzialność za przygotowanie i buforowanie danych w całości spoczywa na mikrokontrolerze. Z racji dużego poboru mocy przez układ radiowy [TODO: Wstaw pobór mocy], linia zasilania, podobnie jak w przypadku magistrali I^2C , została wyposażona w odcinający przepływ prądu tranzystor i konwerter stanów logicznych dla linii danych, jak pokazano na rysunku [TODO: Wstaw rysunek].

4.2 Elementy pomiarowe

4.2.1 Barometr, higrometr i termometr

Urządzenie zostało wyposażone w 2 moduły znajdujące się na wspólnej płycie:

- BMP280 — mogący mierzyć temperaturę i ciśnienie atmosferyczne;
- AHT20 — mogący mierzyć temperaturę i wilgotność względną.

Obydwa moduły używają magistrali I^2C do komunikacji. [TODO: Rozwiń]

4.2.2 Moduł z licznikiem Geigera-Müllera

Moduł detekcji promieniowania jonizującego wykorzystuje licznik Geigera-Müllera oparty na projekcie opublikowanym na forum Elektroda [2], zmodyfikowanym na potrzeby niniejszej pracy przez inż. Karola Karpowicza (rysunek [TODO: Wstaw zdjęcie]). Układ składa się z licznika Geigera-Müllera typu J321, wzmacniacza impulsów

oraz generatora wysokiego napięcia. W chwili detekcji cząstki promieniowania na wyjściu modułu generowany jest impuls, który podawany jest na wejście licznikowe mikrokontrolera. Implementacja algorytmu zliczania impulsów została omówiona w rozdziale [5.3.4](#).

4.2.3 Moduł pomiaru natężenia światła

Za pomiar natężenia światła odpowiedzialny jest moduł VEML7700. Podobnie jak BMP280 i AHT20 komunikuje się z mikrokontrolerem używając protokołu I^2C . Natężenie światła mierzone jest w luksach. [TODO: Rozwiń?]

4.3 Elementy dodatkowe

4.3.1 Magistrala 1-Wire

[Nie zaimplementowane]

4.3.2 Wolne wyprowadzenia GPIO

[Nie zaimplementowane]

Rozdział 5

Oprogramowanie

5.1 Język Rust w programowaniu mikrokontrolerów

5.2 Narzędzia i biblioteki

5.3 Obsługa elementów pomiarowych

W celu ograniczenia wykorzystania zasobów pamięciowych mikrokontrolera znaczna część konfiguracji urządzeń pomiarowych realizowana jest na etapie kompilacji. Rozbudowany system typów (ang. *type system*) języka Rust umożliwia zwięzłe i czytelne wyrażenie konfiguracji z wykorzystaniem typów generycznych (ang. *generic types*), cech (ang. *traits*) oraz stałych czasu kompilacji (ang. *const generics*).

5.3.1 Moduł AHT20 — czujnik temperatury i wilgotności względnej

Obsługa modułu AHT20 została zrealizowana w postaci struktury `Aht20` wraz z jej implementacją. Komunikacja realizowana jest za pośrednictwem magistrali I²C poprzez wysyłanie dedykowanych komend sterujących. Moduł nie wymaga przesyłania danych konfiguracyjnych w trakcie działania. Zgodnie z notą katalogową producenta [5] urządzenie obsługuje następujący zestaw komend:

- **Inicjalizacja** — wymuszenie procedury kalibracji wewnętrznej układu;
- **Wyzwolenie pomiaru** — zainicjowanie akwizycji danych oraz ich zapis do wewnętrznej pamięci urządzenia;
- **Odczyt statusu** — weryfikacja stanu kalibracji układu oraz gotowości do przeprowadzenia kolejnego pomiaru.

Procedura inicjalizacji modułu, przedstawiona na listingu 5.1, rozpoczyna się od odczekania czasu rozruchu wynoszącego 40ms, po którym następuje odczyt rejestru statusu. W przypadku braku flagi kalibracji wysyłana jest komenda inicjalizacji, a następnie wprowadzane jest opóźnienie 10ms niezbędne do jej wykonania:

Listing 5.1: Inicjalizacja modułu AHT20

```
1 pub fn init<D: DelayNs>(
2     &self, i2c: &mut I2c,
3     delay: &mut D
4 ) -> Result<(), Aht20Error> {
5     delay.delay_ms(40);
6
7     let status = self.read_status(i2c)?;
8
9     if status & STATUS_CALIBRATED_BIT == 0 {
10         i2c.write(self.address, &COMMAND_INITIALIZE)?;
11         delay.delay_ms(10);
12     }
13
14     Ok(())
15 }
```

Odczyt surowych danych pomiarowych, przedstawiony na listingu 5.2, realizowany jest w trzech etapach. W pierwszym kroku wysyłana jest komenda wyzwolenia pomiaru, po której, zgodnie z wymaganiami noty katalogowej, następuje odczekanie 80ms do czasu jego zakończenia. Następnie odczytywany jest rejestr statusu w celu weryfikacji gotowości układu — jeżeli bit zajętości jest ustawiony, funkcja zwraca błąd `Aht20Error::SensorBusy`. W przypadku pomyślnej weryfikacji dane pomiarowe zostają zaczytane do 7-bajtowego bufora i zwrócone w postaci struktury `Aht20RawData`. [TODO: może skrócić? Bo wsm to opis kodu]

Listing 5.2: Odczyt surowych danych z modułu AHT20

```
1 pub fn read_raw<I: I2c, D: DelayNs>(
2     &self,
3     i2c: &mut I,
4     delay: &mut D,
5 ) -> Result<Aht20RawData, Aht20Error> {
6     i2c.write(self.address, &COMMAND_START_MEASUREMENT)?;
7 }
```

```
8 // Datasheet 5.4 step 3: wait 80 ms, then check busy bit
9 delay.delay_ms(80);
10
11 let status = self.read_status(i2c)?;
12 if status & STATUS_BUSY_BIT != 0 {
13     return Err(Aht20Error::SensorBusy);
14 }
15
16 let mut bytes = [0u8; 7];
17 i2c.read(self.address, &mut bytes)?;
18
19 Ok(Aht20RawData { bytes })
20 }
```

Konwersja surowych danych odczytanych z modułu na stopnie Celsjusza oraz wilgotność względną jest możliwa przy wykorzystaniu wzorów dostarczonych w nocie katalogowej [5]. Dla temperatury podano wzór:

$$T [^{\circ}\text{C}] = \frac{S_T}{2^{20}} \cdot 200 - 50 \quad (5.1)$$

gdzie:

- T — temperatura w stopniach Celsjusza,
- S_T — surowa wartość temperatury odczytana z czujnika.

Dla wilgotności względnej podano wzór:

$$RH [\%] = \frac{S_{RH}}{2^{20}} \cdot 100\% \quad (5.2)$$

gdzie:

- RH — wilgotność względna,
- S_{RH} — surowa wartość wilgotności względnej odczytana z czujnika.

Funkcje odpowiedzialne za konwersję przedstawiono na listingu 5.3. Wzory (5.1) i (5.2) zostały przekształcone w sposób niewymagający arytmetyki zmiennoprzecinkowej — dzielenie przez 2^{20} zastąpiono przesunięciem bitowym w prawo o 20 pozycji i wykonuje się ono dopiero po przemnożeniu surowej wartości. Wprowadzono ponadto stałą czasu kompilacji `PRECISION`, pozwalającą na skalowanie wyników i ograniczenie strat wynikających z arytmetyki całkowitoliczbowej. Przekazanie wartości `PRECISION = 1000` skutkuje zwróceniem wyników w m°C dla funkcji `convert_temperature` oraz w tysięcznych częściach procenta dla funkcji `convert_humidity`.

Listing 5.3: Konwersja surowych danych z modułu AHT20

```

1 pub fn convert_temperature<const PRECISION: i64>(raw: i64) -> i16 {
2     (((raw * 200 * PRECISION) >> 20) - 50 * PRECISION) as i16
3 }
4
5 pub fn convert_humidity<const PRECISION: i64>(raw: i64) -> i16 {
6     ((raw * 100 * PRECISION) >> 20) as i16
7 }

```

Integralność odebranych danych może zostać zweryfikowana za pomocą 8-bitowej sumy kontrolnej CRC, znajdującej się w siódmym bajcie bufora odczytu. Weryfikacja realizowana jest przez funkcję `check_crc`, przedstawioną na listingu 5.4. [TODO: Dopytaj czy opis potrzebny]

Listing 5.4: Sprawdzanie sumy kontrolnej danych modułu AHT20

```

1 fn check_crc(bytes: &[u8; 7]) -> bool {
2     let mut crc: u8 = 0xFF;
3     for &byte in &bytes[..6] {
4         crc ^= byte;
5         for _ in 0..8 {
6             if crc & 0x80 != 0 {
7                 crc = (crc << 1) ^ 0x31;
8             } else {
9                 crc <<= 1;
10            }
11        }
12    }
13    crc == bytes[6]
14 }

```

Implementacja struktury `Aht20` została uzupełniona o funkcję `read` — przedstawioną na listingu 5.5 — wykonującą surowy pomiar, weryfikację integralności danych oraz ich konwersję. Surowe wartości temperatury i wilgotności są 20-bitowymi liczbami całkowitymi bez znaku, które przed przetwarzaniem zostają rozszerzone do typu `i64`. Wybór tego typu podyktowany jest koniecznością uniknięcia przepełnienia podczas pośrednich operacji arytmetycznych. Należy zaznaczyć, że typ `i64` nie jest natywnie wspierany przez architekturę AVR i musi być realizowany programowo, co negatywnie wpływa na wydajność kodu. Wartości zwracane przez funkcję podane są z dokładnością do setnych części procenta i stopnia Celsjusza, opakowane wraz z flagą integralności da-

nych w strukturę `Aht20Data`. [TODO: może nie dodawaj całości funkcji tylko ciekawe fragmenty]

Listing 5.5: Odczyt danych modułu AHT20

```
1 pub fn read<D: DelayNs>(  
2     &self, i2c: &mut I2c,  
3     delay: &mut D  
4 ) -> Result<Aht20Data, Aht20Error> {  
5     let raw = self.read_raw(i2c, delay)?;  
6  
7     let crc_passed = Self::check_crc(&raw.bytes);  
8  
9     // Humidity: bits [39:20] of the data payload (bytes 1-3)  
10    let raw_humidity = 0i64  
11        | ((raw.bytes[1] as i64) << 12)  
12        | ((raw.bytes[2] as i64) << 4)  
13        | ((raw.bytes[3] as i64) >> 4);  
14  
15    // Temperature: bits [19:0] of the data payload (bytes 3-5)  
16    let raw_temperature = 0i64  
17        | ((raw.bytes[3] as i64 & 0x0F) << 16)  
18        | ((raw.bytes[4] as i64) << 8)  
19        | (raw.bytes[5] as i64);  
20  
21    Ok(Aht20Data {  
22        temperature:  
23            Self::convert_temperature::<100>(raw_temperature),  
24        humidity:  
25            Self::convert_humidity::<100>(raw_humidity),  
26        crc_passed,  
27    })  
28 }
```

5.3.2 Moduł BMP280

Moduł BMP280 jest obsługiwany przez strukturę `Bmp280`, pokazaną na listingu 5.6. Zawiera ona pole adresu urządzenia na magistrali I²C, pole przechowujące dane kompensacyjne pomiaru oraz parametr typu `CONFIG` zawarty w polu `PhantomData` o zero-

wym rozmiarze w pamięci.

Listing 5.6: Struktura Bmp280

```

1 pub struct Bmp280<CONFIG>{
2     address: u8,
3     compensations: Option<Bmp280Compensations>,
4     _config: PhantomData<CONFIG>,
5 }

```

Dane kompensacyjne zapisywane są do struktury `Bmp280Compensations`, składającej się z dwunastu szesnastobitowych pól przechowujących wartości kompensacyjne. Są one wykorzystywane przy przeliczaniu surowych danych odczytanych z modułu na temperaturę w stopniach Celsjusza oraz ciśnienie w paskalach. Implementacja struktury udostępnia funkcję powiązaną `read_from_i2c`, odczytującą 24 bajty z pamięci modułu BMP280 i przypisującą je do poszczególnych pól struktury. Implementacja tej funkcji przedstawiona jest na listingu 5.7, a nazwy pól struktury przyjęto zgodnie z notą katalogową [8].

Listing 5.7: Odczyt danych kompensacyjnych dla modułu BMP280

```

1 pub fn read_from_i2c(address: u8, i2c: &mut I2c)
2     -> Result<Self, i2c::Error>{
3
4     let mut buffer = [0u8; N_COMPENSATION_BYTES];
5
6     i2c.write_read(
7         address,
8         &[COMPENSATION_START_ADDRESS],
9         &mut buffer
10    )?;
11
12    Ok(Self{
13        dig_t1: u16::from_le_bytes([buffer[0], buffer[1]]),
14        dig_t2: i16::from_le_bytes([buffer[2], buffer[3]]),
15        dig_t3: i16::from_le_bytes([buffer[4], buffer[5]]),
16
17        dig_p1: u16::from_le_bytes([buffer[6], buffer[7]]),
18        // ...
19        dig_p9: i16::from_le_bytes([buffer[22], buffer[23]]),
20    })

```

21 }

Konfiguracja modułu BMP280 odbywa się poprzez zapis 8-bitowych wartości do rejestrów CTRL_MEAS i CONFIG. Wartości te pozyskiwane są z typu implementującego cechę Config, widoczną na listingu 5.8. Cecha ta definiuje skojarzone typy odpowiedzialne za poszczególne ustawienia modułu:

- TempOversampling – określa stopień nadpróbkowania pomiaru temperatury,
- PressOversampling – określa stopień nadpróbkowania pomiaru ciśnienia,
- PowerMode – określa tryb pracy urządzenia,
- Standby – określa odstęp czasowy pomiędzy pomiarami,
- IIR – określa stałą filtra IIR.

Każdy z tych typów ograniczony jest do odpowiedniej cechy udostępniającej stałą BITS, reprezentującą wartość bitową zgodną z notą katalogową. Wymagane wartości rejestrów przechowywane są w stałych CTRL_MEAS_VALUE i CONFIG_VALUE, których wartości powstają przez przesunięcie bitowe stałych BITS na odpowiednie pozycje i złożenie ich operatorem alternatywy bitowej. Rejestr CONFIG_VALUE dodatkowo określa protokół komunikacyjny urządzenia – pole EN_SPI ustawione jest na 0, co wymusza użycie protokołu I²C.

Listing 5.8: Cecha Config używana do konfiguracji BMP280

```
1 pub trait Config{
2     type TempOversampling: Oversampling;
3     type PressOversampling: Oversampling;
4     type PowerMode: PowerMode;
5     type Standby: Standby;
6     type IIR: IIRConstant;
7
8     const CTRL_MEAS_VALUE: u8 =
9         Self::TempOversampling::BITS << TEMP_OVERSAMPLING_OFFSET
10        | Self::PressOversampling::BITS << PRESS_OVERSAMPLING_OFFSET
11        | Self::PowerMode::BITS << POWER_MODE_OFFSET;
12
13     const CONFIG_VALUE: u8 =
14         Self::Standby::BITS << STANDBY_OFFSET
15        | Self::IIR::BITS << IIR_CONSTANT_OFFSET
16        // Disable SPI mode
```

```

17         | 0b0 << EN_SPI_OFFSET;
18     }
19
20 pub trait Oversampling{ const BITS: u8; }
21 pub trait PowerMode{ const BITS: u8; }
22 pub trait Standby{ const BITS: u8; }
23 pub trait IIRConstant{ const BITS: u8; }

```

Przykładową implementację cechy `Config` oraz cech odpowiedzialnych za poszczególne wartości bitowe przedstawiono na listingu 5.9. Definicje pomocniczych cech `Oversampling`, `PowerMode`, `Standby` i `IIRConstant` zostały pominięte, ponieważ są już widoczne na listingu 5.8.

Listing 5.9: Przykładowa implementacja cech konfiguracyjnych modułu BMP280

```

1 pub struct DefaultConfig;
2
3 impl Config for DefaultConfig{
4     type TempOversampling = OversamplingX1;
5     type PressOversampling = OversamplingX2;
6     type PowerMode = PowerModeNormal;
7
8     type Standby = Standby0_5ms;
9     type IIR = IIROff;
10 }
11
12 pub struct OversamplingX1;
13 pub struct OversamplingX2;
14 impl Oversampling for OversamplingX1{ const BITS: u8 = 0b001; }
15 impl Oversampling for OversamplingX2{ const BITS: u8 = 0b010; }
16
17 pub struct PowerModeNormal;
18 impl PowerMode for PowerModeNormal{ const BITS: u8 = 0b11; }
19
20 pub struct Standby0_5ms;
21 impl Standby for Standby0_5ms{ const BITS: u8 = 0b000; }
22
23 pub struct IIROff;
24 impl IIRConstant for IIROff{ const BITS: u8 = 0b000; }

```

5.3.3 Moduł VEML7700

5.3.4 Moduł z licznikiem Geigera-Müllera

Obsługa licznika odbywa się za pośrednictwem struktury `GeigerCounter`. Sprzętowy licznik T1 mikrokontrolera skonfigurowany jest do zliczania impulsów zewnętrznych generowanych przez lampę [AI mówi rurę] Geigera-Müllera. Co sekundę zliczona wartość jest odczytywana i zapisywana do 60-elementowego bufora cyklicznego.

Funkcja `cpm` zwraca liczbę zliczeń na minutę poprzez ekstrapolację zsumowanych wartości bufora na pełną minutę, nawet w przypadku niepełnego zapełnienia bufora. Takie podejście ma dwie konsekwencje: przy czasie pracy krótszym niż minuta wynik może być obciążony większym błędem, natomiast przy pełnym buforze nagłe skoki wartości są wygładzane przez uśrednianie.

Kod odpowiedzialny za odczyt i reset licznika T1 przedstawiono na listingu 5.10. W celu zabezpieczenia przed przerwaniem wykonywania kodu zastosowano funkcję `interrupt::free` z biblioteki `avr_device`.

Listing 5.10: Odczyt zliczeń licznika T1 mikrokontrolera

```
1 fn timer1_read_and_reset(tc1: &TC1) -> u16{
2     avr_device::interrupt::free(|_| unsafe {
3         let val = tc1.tcnt1().read().bits();
4         tc1.tcnt1().write(|w| w.bits(0));
5         val
6     })
7 }
```

5.4 Komunikacja radiowa

5.4.1 Serializacja danych

W celu ustandaryzowania danych zebranych z modułów pomiarowych zostają one opakowane w strukturę `TypedLabeledReadout`.

5.4.2 Zabezpieczenie danych

[Nie zaimplementowane]

5.4.3 Transmisja danych

Rozdział 6

Testy

6.1 Zasięg komunikacji radiowej

6.2 Pobór prądu

6.3 Czas pracy

6.4 Porównanie z rozwiązaniami komercyjnymi

Podsumowanie

Tu treść podsumowania (którą — oczywiście — też piszemy na końcu).

Spis listingów

5.1	Inicjalizacja modułu AHT20	26
5.2	Odczyt surowych danych z modułu AHT20	26
5.3	Konwersja surowych danych z modułu AHT20	28
5.4	Sprawdzanie sumy kontrolnej danych modułu AHT20	28
5.5	Odczyt danych modułu AHT20	29
5.6	Struktura <code>Bmp280</code>	30
5.7	Odczyt danych kompensacyjnych dla modułu BMP280	30
5.8	Cecha <code>Config</code> używana do konfiguracji BMP280	31
5.9	Przykładowa implementacja cech konfiguracyjnych modułu BMP280 . .	32
5.10	Odczyt zliczeń licznika T1 mikrokontrolera	33

Spis tabel

1.1	Zestawienie wybranych modułów pomiarowych dostępnych na rynku . .	8
2.1	Porównanie języków programowania w systemach wbudowanych.	14
4.1	Napięcie wyjściowe U_{wy} regulatora bocznikowego przy użyciu diody Zenera i diody LED, dla prądu emitera I_E tranzystora zwierającego. Do testu użyto zasilacza stałoprądowego.	19
4.2	Opis funkcji wyprowadzeń mikrokontrolera. Typy portów: I/O - cyfrowy port wejścia/wyjścia, Tn - podłączenie do wewnętrznego licznika n mikrokontrolera, ADC - wejście konwertera analogowo-cyfrowego. . . .	20

Spis rysunków

4.1	Schemat elektryczny części sprzętowej projektu.	17
4.2	Sekcja ładowania superkondensatorów z użyciem diod zenera (D1, D2) . .	18
4.3	Sekcja ładowania superkondensatorów z użyciem diod LED (D1, D2) . .	18
4.4	Układ regulacji napięcia oparty o przetwornicę step-up i regulator liniowy.	19
4.5	Mikrokontroler ATmega328P widoczny na schemacie projektu.	21
4.6	Tranzystor załączający linię zasilania urządzeń magistrali I^2C	22
4.7	Konwertery poziomów logicznych dla linii SDA i SCL magistrali I^2C . .	22

Bibliografia

- [1] Avr uart baud rate calculator.
- [2] Anonimowy użytkownik forum Elektroda.pl. Licznik geigera (bez zniekształconego MC34063) o poborze prądu $< 3\text{mA}$. <https://www.elektroda.pl/rtvforum/topic3582237.html>, 2026. Forum internetowe Elektroda.pl; dostęp: 26 maja 2026.
- [3] Arduino. Arduino uno rev3.
- [4] Arduino. Nano | arduino documentation.
- [5] Asair - Guangzhou Aosong Electronics Co., Ltd. *AHT20 Product manuals*, 2019.
- [6] Atmel Corporation. *ATmega328P*, 2015.
- [7] Michael Barr and Anthony Massa. *Programming Embedded Systems: With C and GNU Development Tools*. O'Reilly Media, 2nd edition, 2006.
- [8] Bosch Sensortec GmbH. *BMP280 Digital Pressure Sensor – Data Sheet*. Bosch Sensortec GmbH, revision 1.26 edition, October 2021. Dostęp: 2026-05-31.
- [9] Holtek Semiconductor Inc. *HT73XX Low Power Consumption LDO*.
- [10] Marcy Lowe, Saori Tokuoka, Tali Trigg, and Gary Gereffi. Lithium-ion batteries for electric vehicles: the u.s. value chain. 10 2010.
- [11] Alain Nakad, Mervat Madi, Olav Aaker, Efstratios Ntantis, and Karim Kabalan. Comparing supercapacitors to lithium-ion batteries through measurements and simulations. volume 2023, 11 2023.
- [12] Nanjing Micro One Electronics Inc. *DC/DC Step up Converter ME2108 Series*.
- [13] Emmanuel Pamet , Lukas K ps, Fabian Alexander Kreth, Sebastian Pohlmann, Alberto Varzi, Thierry Brousse, Andrea Balducci, and Volker Presser. The many deaths of supercapacitors: Degradation, aging, and performance fading. *Advanced Energy Materials*, 13(29):2301008, 2023.

- [14] NXP Semiconductors. *UM10204 I2C-bus specification and user manual*. NXP Semiconductors, 10 2021.